Ders 2 Çalışma Özeti

Ders 2 Çalışma Özeti

Genel Konular

- Bu kayıt uygulama (laboratuvar) saatidir.
- İlk derste sorulan düşünme sorusu birlikte çözülür: doğrusal yoklamalı (linear probing) bir hash tablosunda elemanların ortalama kaçıncı denemede yerleştiğinin hesaplanması.
- Çözüm üç aşamada sadeleştirilir: önce `if` ile üç durum, sonra iki durum, en sonda tek bir `mod` ifadesi.
- Geçen dönemki Algoritma Analizi sınavında öğrencilerin çoğunun verdiği verimsiz çözüm incelenir ve neden kötü olduğu tartışılır.
- Akış diyagramında `for` ve `while` döngülerinin giriş ve çıkış okları hatırlatılır.

Hocanın Özellikle Vurguladığı Kısımlar

- Bir algoritmanın doğru çalışması yetmez; verimli (efektif) olması da gerekir.
- Çözüme başlamadan önce küçük bir sayısal örnek üzerinde deneme yapın. Problem çoğu zaman gözle görülür.
- Uç noktalara (edge case) dikkat edin. Bu soruda uç durum, tablonun sonundan başa dönerek yerleşen elemandır (örnekte 29).
- Bir elemanın yerini zaten biliyorsanız onu tekrar aramayın.
- Sonucu her zaman doğru olacağını bildiğiniz bir koşulu ayrıca kontrol etmeyin.
- Sınavda en verimli çözümü bulma stresine kapılmayın. Ama çözüm belli bir eşiğin altında kötü olmamalıdır. Bir iki dakika düşünüp çözümünüzü sadeleştirin.

Kısa Tekrar Notları

- Tablo boyutu N, eleman x ise ev adresi (home address) x mod N'dir.
- Göz doluysa sıradaki gözlere bakılır. Tablonun sonuna gelinirse başa (0. göze) dönülür.
- `table[i]` gözündeki elemanın ev adresi `value = table[i] % N` ise kaçıncı denemede yerleştiği:
  - $step = (N + i - value + 1) \bmod N$
- Ortalama adım sayısı = toplam adım / dolu göz sayısı.
- Boş gözler için hesap yapılmaz. Dolu göz sayısı da sayılmalıdır, yoksa ortalama alınamaz.
- Hashing'in amacı aramayı eleman sayısından bağımsız, birkaç adımda yapmaktır. İkili aramada (binary search) bile karmaşıklık $O(\log n)$'dir.

Detaylı Açıklamalar

Sorunun hatırlatılması

- $N = 10$ gözlü bir tabloya sırasıyla 23, 4, 3, 9, 29 yerleştirilir.
- Yerleşme kuralı: x mod 10 adresine bak; doluysa bir sonraki gözü dene; sondan sonra başa dön.
- Sonuç tablo:

Göz	Eleman	Ev adresi	Kaçıncı denemede yerleşti
0	29	9	2 (9 dolu, başa döndü)

Göz	Eleman	Ev adresi	Kaçıncı denemede yerleşti
3	23	3	1
4	4	4	1
5	3	3	3 (3 ve 4 dolu)
9	9	9	1

- Toplam adım $1 + 1 + 3 + 1 + 2 = 8$. Ortalama $8/5 = 1{,}6$ adım.
- Soruda elinizde yalnızca bu tablo vardır. Elemanlar yerleştirilirken adım sayılmamıştır. Hesap tablodan yapılmalıdır.

Öğrenci önerileri

- Bir öğrenci istatistiksel bir tahminle (en az ve en fazla adımın ortalaması) cevap vermeye çalışır.
  - Hoca bunun örnekle tutmadığını gösterir. Burada istenen gerçek ortalamanın hesaplanmasıdır.
- Bir başka öğrenci şu fikri önerir: elemanın ev adresini bul, bulunduğu gözle arasındaki uzaklığa bak.

Çözümün aşama aşama sadeleştirilmesi

- **1. aşama (üç durum):**
  - `value == i` ise eleman ilk denemede yerleşmiştir: `step = 1`.
  - `value < i` ise eleman aşağı doğru kaymıştır: `step = i - value + 1`.
  - `value > i` ise eleman sondan başa dönmüştür: `step = N + i - value + 1`.
  - Örnek: 5. gözdeki 3 için $5 - 3 + 1 = 3$. 0. gözdeki 29 için $10 + 0 - 9 + 1 = 2$.
  - +1'in nedeni: ilk yerleşme denemesi de bir adımdır.
- **2. aşama (iki durum):**
  - `value == i` kontrolüne gerek yoktur. `value <= i` dalı bu durumu da doğru hesaplar: $3 - 3 + 1 = 1$.
- **3. aşama (tek ifade):**
  - Bir öğrencinin önerisi: iki dalda da N ekle, sonra N'ye göre mod al.
  - $step = (N + i - value + 1) \bmod N$
  - 5. gözdeki 3: $(10 + 5 - 3 + 1) \bmod 10 = 13 \bmod 10 = 3$.
  - 0. gözdeki 29: $(10 + 0 - 9 + 1) \bmod 10 = 2$.
  - Tabloya sonradan 39 gelirse: 9 ve 0 dolu olduğu için 1. göze yerleşir. $(10 + 1 - 9 + 1) \bmod 10 = 3$.
- Hoca üç aşamanın da doğru ve kabul edilebilir olduğunu söyler. Son aşama en kısa olanıdır.

Son algoritma (ekrandan, 00:29:35)

```
N, table[N] oku
sum = 0
cnt = 0
i = 0'dan N-1'e kadar:
    eğer table[i] != NULL (göz dolu) ise:
        value = table[i] % N
        step  = (N + i - value + 1) % N
        cnt++
        sum += step
average = sum / cnt
```

- Ders sırasında bir öğrenci, `value` hesaplanmadan `step` yazıldığını fark eder ve hoca düzeltir.

C karşılığı

Derste C kodu yazılmadı. Aşağıdaki kod, ekrandaki akış diyagramından ajan tarafından çevrilmiştir. `gcc -Wall` ile derlenip yukarıdaki örnek tabloyla çalıştırılmıştır. Çıktı 1.60'tır; 39 eklenmiş tabloda 1.83'tür.

```c
#include <stdio.h>

#define BOS 0   /* boş göz; örnekte tablo baştan sıfırlanmış kabul edilir */

/* Doğrusal yoklamalı tabloda elemanların ortalama kaçıncı denemede yerleştiğini hesaplar. */
double ortalama_adim(int table[], int n) {
    int i, value, step, sum = 0, cnt = 0;
    for (i = 0; i < n; i++) {
        if (table[i] != BOS) {
            value = table[i] % n;              /* elemanın ev adresi */
            step = (n + i - value + 1) % n;    /* kaçıncı denemede yerleşti */
            cnt++;
            sum += step;
        }
    }
    return (double)sum / cnt;
}

int main(void) {
    int t1[10] = {29, BOS, BOS, 23, 4, 3, BOS, BOS, BOS, 9};
    printf("%.2f\n", ortalama_adim(t1, 10));
    return 0;
}
```

Verimsiz (kötü) çözüm örneği

- Geçen dönemki Algoritma Analizi sınavında öğrencilerin büyük çoğunluğu şöyle bir çözüm yazmıştır (ekrandan, 00:35:10). Ekranda koşul `table[adr] != value` olarak yazılıdır. Hoca bunu, `table[i]` elemanının kendisinin yeniden arandığı şeklinde anlatır:

```
value = table[i] % N
eğer value == i ise:
    step = 1
değilse:
    cnt = 0
    adr = value
    (cnt < n) VE (table[adr] != aranan eleman) olduğu sürece:
        adr = (adr + 1) % N
        cnt++
```

- Yani eleman, ev adresinden başlanarak tabloda yeniden aranır ve adımlar sayılır.
- Hocanın eleştirileri:
  - Eleman zaten `table[i]` gözünde bulunmuştur. Yerini bildiğiniz bir elemanı tekrar aramanın anlamı yoktur.
  - `cnt < n` kontrolü, "eleman tabloda yoksa" durumu içindir. Ama eleman tabloda vardır, arama mutlaka biter. Bu kontrol gereksizdir.
  - Bu kontrol, ilk derste doğrusal arama için konuşulan "eleman dizide olmayabilir" kontrolünün yanlış yere taşınmasıdır.
- Hoca bu çözümlere de puan verildiğini, ama çözümün verimsizliğini göstermek için paylaştığını söyler.

Hashing neden kullanılır

- Bilgisayar bilimlerinde en çok uğraşılan işlemlerden biri aramadır.
- Hashing, elemana eleman sayısı n'den bağımsız olarak birkaç adımda erişmeyi hedefler.

- Tablo boyutu eleman sayısının yaklaşık iki katı seçilirse aranan elemana en fazla birkaç (3-4) adımda ulaşılır.
- Bunun istatistiksel dayanağı doğum günü paradoksu (birthday paradox) mantığıyla açıklanır. Ayrıntısı 3. sınıftaki Algoritma Analizi dersinde işlenir.

Akış diyagramında döngü okları

- `for` kutusunda: yukarıdan girilir, döngü gövdesinden sonra kutuya geri dönülür, döngü bitince kutunun çıkış okundan devam edilir.
- `while` (karar kutusu) için: koşul doğruysa (T) gövde yapılır ve koşula geri dönülür; koşul yanlışsa (F) döngüden çıkılır.

Ders sonrası notlar

- Hoca tahtada bir ara M ve N harflerini karıştırır ve düzeltir. Formüldeki tablo boyutu her yerde N'dir.
- $step = (N + i - value + 1) \bmod N$ ifadesi, eleman tam N. denemede yerleşmişse 0 verir. Bu durum yalnızca tablonun tamamen dolu olduğu uç durumda ortaya çıkabilir. Tablo yarı yarıya dolu tutulduğunda bu durum oluşmaz. Her durumda doğru sonuç için $step = (N + i - value) \bmod N + 1$ yazılabilir (ajan notu).
- C örneğindeki gibi boş gözü 0 ile gösterirseniz, $x = 0$ değeri tabloya konamaz. Gerçek uygulamada boş göz için ayrı bir işaret (ör. -1 veya ayrı bir dolu/boş dizisi) kullanılır.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.